WONDER BOX — SYSTEM ARCHITECTURE

Objective: Zero confusion. Zero architectural drift.

CORE DOCTRINE (NON-NEGOTIABLE)
• This is a finite state machine
• UI = pure function of state
• Hardware & AI are services, not logic
• Every failure collapses to ERROR
• Logging is first-class

If any of these are violated, the system becomes un-debuggable.

MENTAL MODEL
Wonder Box is a deterministic embedded system driven by states.
Think aircraft software, not a web app.

CHILD POV FLOW
IDLE → LISTENING → PROCESSING → DISPLAYING → PRINTING → IDLE
Any failure: ANY STATE → ERROR → IDLE

REPOSITORY STRUCTURE
(main.py, core/, ui/, services/, hardware/, utils/, logs/)

ARCHITECTURAL RULES
State Machine:
- Owns state and transitions
- No UI, hardware, or AI code

UI Layer:
- Renders based on state only
- Emits intent via callbacks

Services:
- Perform side-effects
- Never change state directly

Hardware:
- Abstract GPIO and devices
- No business logic

ERROR PHILOSOPHY
Single strategy:
Fail → Log → ERROR → Reset → IDLE

LOGGING
• Timestamp
• State
• Event
• Details
• Error

DEVELOPMENT RULES
- No feature without a state
- No state without a screen
- No service without logging

ENVIRONMENT
Python 3.11+, virtualenv only.

FINAL NOTE
If tempted to hack:
Stop. Fix architecture instead.

This document is law.